

The Story of How We Make the Spark

Designing the seed-generation layer (Leg 1) without a VRF,
using only hash functions and public beacons

A friendly reading guide. This is the companion to the certified-randomness math story. That document told the tale of how a quantum measurement becomes certified random bits; this one tells the tale of the step that comes before all of that — how we make the small, fresh, unpredictable starting value (the seed) that triggers each round. The seed is the spark that lights the rest of the engine. If the spark is rigged, everything downstream is compromised, no matter how clever the math is. So this is, in its own way, the most foundational chapter of all.

Prologue: a spark that nobody can rig

Picture the whole certified-randomness protocol as a small factory that turns a tiny seed of unpredictability into a large bag of certified random bits. Every round, the factory needs a fresh starting value to feed in — a short string of random bytes that tells the rest of the machinery which puzzles to ask, which circuits to generate, which experiments to run. That starting value is called the **seed**.

The seed has to be three things at once:

1. **Unpredictable.** If the quantum server can guess the seed in advance, they can pre-compute answers to the circuits the seed will generate, and submit those pre-computed answers instead of doing real quantum measurements. (We saw earlier why this matters: the deadline-based proof of quantumness collapses if the server can do the work *before* we send the request.)
2. **Unbiasable.** No single party — not us, not the server, not any external observer — should be able to nudge the seed toward a value of their choosing. Otherwise that party can rig the certified randomness in their favour.
3. **Verifiable.** Anyone — the client, an auditor, a regulator, a customer reading the audit log a year later — should be able to independently confirm that the seed was honestly generated and not rigged.

The challenge is that we run the protocol as a *deployed multi-party service*. We have an operator (us), we have miners (vendors who run the quantum hardware), we have customers (whoever consumes the bits), and none of these parties fully trusts the others. So the seed has to be *trustless* — secure even when some parties are dishonest. Which means we can't just say "our operator generates the seed and we promise it's honest." Anyone could doubt that promise.

So: how do you make a number that's small, fresh, unpredictable, unbiasable, verifiable, and

trustless? The textbook answer is a primitive called a Verifiable Random Function. We tried it. It turned out to have problems. This is the story of what those problems were, and how we now plan to solve the same task without it.

Cast of characters

Operator. The party running the protocol (us). Has its own cryptographic identity. Can contribute to the seed but *cannot* unilaterally produce it.

Public contribution pool. An open inbox. Anyone in the world can drop a contribution into it during each round’s submission window.

drand beacon. An external, decentralised public randomness service run by a federation including Cloudflare, EPFL, Protocol Labs, and others. Publishes a new random “pulse” every 30 seconds, signed with threshold cryptography.

NIST randomness beacon. An external, centralised public randomness service run by the U.S. National Institute of Standards and Technology. Publishes a new signed random pulse every 60 seconds.

Verifier. Anyone who wants to check that a particular round’s seed was honestly generated. Could be the client, an auditor, a customer reading the audit log, a regulator.

Adversary. Worst-case bad actor. Could be the operator, a pool contributor, both, or an external attacker. We assume they’re sophisticated and well-funded.

Round. One run of the protocol. Indexed by an integer R . Round R produces seed seed_R .

1 Chapter 1: the textbook answer — and why we tried it

When we first sat down to design Leg 1, the textbook cryptographic tool for “produce a verifiable random value tied to a public input, in a way no single party can rig” is a **Verifiable Random Function**, or VRF. A VRF is a cryptographic gadget with two bundled jobs:

1. **Produce a pseudo-random output** from a secret key and a public input.
2. **Produce a proof** that anyone can check (using only the corresponding public key, the input, and the output) to confirm the output was generated honestly.

VRFs are everywhere: Chainlink VRF, Algorand’s leader selection, DNSSEC’s NSEC5 mechanism. They’re the standard answer to “produce trustless verifiable randomness.” So our v0.2 implementation used **EC-VRF**, the elliptic-curve VRF based on Ed25519, which is the most widely deployed VRF construction. It works, the math is sound, and there’s a published RFC (RFC 9381) standardising it.

That seemed like the right answer. But on closer reading, it turned out to carry three independent problems — some structural, some workflow-level, some cryptographic — that together made it the wrong fit for a quantum-enabled protocol.

2 Chapter 2: three problems with EC-VRF

Problem 1: it isn't quantum-secure

EC-VRF's security rests on the hardness of a number-theoretic problem called the **elliptic-curve discrete logarithm problem**: given a public key, you can't recover the private key in any reasonable amount of time. On a classical computer, this is currently considered safe.

On a sufficiently capable *quantum* computer, however, the problem is solvable in polynomial time by Shor's algorithm. A quantum attacker could recover the private key from the public key, and from that point forward, every VRF output they wanted to forge would be forgeable. Every signature, every proof, every certified random value — compromised.

This is the deepest of the three problems, and the one with the cleanest moral. We are building a protocol whose entire value proposition is the harvesting of quantum capability. To stand the seeding layer on cryptographic assumptions that the very same quantum capability could one day break is internally inconsistent. We'd be betting the foundation against the technology we're built on top of.

The next natural question is: *couldn't we just use a post-quantum VRF instead?* The answer turns out to be “you could, but it doesn't help you,” for reasons that depend on the next two problems.

Problem 2: VRF is structurally a single-keyholder primitive

This is the most subtle of the three, and worth slowing down on. *Per VRF evaluation*, there is exactly one secret key, held by exactly one party. That's not an implementation choice; it's structural to what a VRF is. The output is computed from (secret key, input); the proof is verifiable against (public key, input, output). If you wanted “many parties' randomness combined,” you'd have to run *many separate VRFs* and combine their outputs.

That sets up a dilemma. To deploy a VRF, you must choose between two paths:

Path A (centralised): One operator holds one key. The protocol runs cleanly and simply. But the operator is now a single point of trust — if compromised or coerced, they can manipulate every seed — and the operator is also exposed to MEV (more on that below).

Path B (decentralised): Many parallel operators each hold their own key. Each round, all of them run their VRFs in parallel and the outputs are combined. The combined seed is honest as long as enough operators are honest.

Path B sounds attractive, but it doesn't come free. To make it actually deliver decentralisation, you have to build a **validator network** around the parallel VRFs: a registration system (who is a valid participant?), Sybil resistance (how do we stop one actor pretending to be a hundred?), an incentive layer (why would honest participants show up every round?), and a slashing layer (what happens when one of them misbehaves?). That's essentially the same kind of system Ethereum's proof-of-stake consensus is, and it's a years-long engineering project of its own.

We'd be standing up an entire mini validator-network just to generate one short random value per round. The cost is wildly out of proportion to the task.

So the VRF abstraction forces a choice between centralisation (with all the trust problems that brings) and a validator network (with all the complexity that brings).

Problem 3: request-then-fulfill is MEV-exposed

This is the workflow-level problem. In any deployment where an external party requests randomness from an operator who then produces and delivers it, there is a **time gap** between “operator sees the request” and “operator publishes the output.” During that gap, the operator has *exclusive knowledge* of what the random value is about to be.

What can they do with this knowledge?

- **Selectively delay.** If the value would be inconvenient (they bet against it, say), they can “lose” the message, blame a network glitch, and let the round expire.
- **Front-run.** They can place trades, bets, or other transactions *based on knowing the upcoming value* before the requester sees it.
- **Reorder.** If they’re also a block-builder (or coordinate with one), they can insert their own transactions ahead of the public fulfillment.
- **Leak.** They can tip off allies who can act on the value before the public delivery.

This kind of attack is called **MEV** (maximum extractable value), and it’s a live, current, well-documented attack class on blockchain oracles. The cryptographic output is fine — the random value, when delivered, is genuinely unpredictable to outsiders and verifiable by them. The attack is on the *workflow*, not the math. **Chainlink VRF is the canonical production example** of this exposure, but the exposure is structural to any operator-served VRF deployment, including our own v0.2 if we deployed it externally.

Why post-quantum VRF doesn’t help

A post-quantum VRF (lattice-based like LB-VRF, or hash-based like X-VRF) changes the *math* under the hood — swaps elliptic curves for lattices or hash chains. That fixes Problem 1: a Shor-equipped quantum attacker can no longer break the underlying primitive.

But Problems 2 and 3 are not cryptographic; they are *architectural* and *workflow-level*. PQ-VRF is still a VRF, still single-keyholder per evaluation, still deployed in a request-then-fulfill pattern. The single-keyholder dilemma is unchanged. The MEV surface is unchanged.

A clean way to hold it: PQ-VRF would solve one of three problems and leave the other two intact. And PQ-VRFs are themselves research-grade — no NIST standard yet, KB-scale proofs, often few-time use or stateful, and at least one (X-VRF) has had its uniqueness proof broken at Financial Cryptography 2024-25. They’re not a ready drop-in replacement; they’re a hopeful research direction. Even if they were mature, they wouldn’t fix the architecture.

So the right move is not to swap to a quantum-safe VRF, but to *drop the VRF abstraction entirely*. We need a different construction — one that doesn’t have a single keyholder, doesn’t deploy in request-then-fulfill, and uses only quantum-resistant primitives. The construction has a name: **multi-source commit-reveal with external beacons**.

3 Chapter 3: the new building blocks

The new design has two ideas working together. Let me introduce them separately first, then show how they combine.

Idea 1: commit-reveal defeats the last-revealer attack

Suppose two parties want to jointly produce a random number by each contributing a value and summing them. The naive approach is: party 1 publishes their value v_1 , then party 2 publishes their value v_2 , then they sum.

The trouble is that party 2, seeing v_1 first, can *choose* v_2 to push the sum wherever they want. They have a privileged last-mover advantage. This is the **last-revealer attack**, and it breaks any naive joint-randomness scheme.

The fix is a two-phase protocol called **commit-reveal**:

Commit phase. Each party picks their value v_i secretly, picks a random nonce n_i , and publishes the **commitment**

$$c_i = H(v_i \parallel n_i)$$

where H is a cryptographic hash function (such as SHAKE-256) and $\parallel$ means byte concatenation. The commitment hides v_i (you can't reverse a hash) but *locks it in* (you can't later claim a different value without the hashes failing to match). All commitments are published *before* anyone reveals anything.

Reveal phase. Once every commitment is publicly recorded, each party publishes their actual (v_i, n_i) . Anyone can re-hash and check that $H(v_i \parallel n_i) = c_i$. If it matches, that party's contribution is accepted. If not, they get excluded.

The intuition: imagine each party writes their number on a piece of paper, seals it inside an opaque envelope, and hands the envelope to a neutral notary. When all envelopes are collected, the notary opens them all at once. Nobody saw anyone else's number while choosing their own; nobody can change their number after the others are revealed. The last-revealer advantage is gone because there is no last revealer in any meaningful sense — the envelopes were all sealed before any opening began.

So commit-reveal is the mechanism that lets multiple contributors safely combine values without anyone gaming the others.

Idea 2: multiple independent source types defeat concentration

Suppose we use commit-reveal with just two contributors: us (the operator) and a public contribution pool. We're better off than with one contributor, but we have a new problem: we've concentrated all the protocol's *liveness* and *censorship* risk into two channels. If either one fails or is blocked, the protocol stalls. If both go silent in the same round, we produce nothing at all.

The fix is to add *independent external sources*. Specifically, we mix in pulses from two public randomness beacons:

- **drand** — a decentralised, threshold-cryptography randomness beacon run by a federation of organisations (Cloudflare, EPFL, Protocol Labs, etc.). Publishes a new pulse every 30 seconds. Honest as long as a majority of the federation is honest.

- **NIST Randomness Beacon** — a centralised but well-known randomness service run by the U.S. National Institute of Standards and Technology. Publishes a new signed pulse every 60 seconds. Honest as long as NIST is honest.

The beacons’ failure modes are **independent** of ours. drand requires colluding a majority of an international consortium; NIST requires compromising a U.S. federal agency; the operator requires compromising us; the public pool requires that *every contributor in the pool* is dishonest. An attacker would have to take down or compromise *all four* channels to halt the protocol or bias the output.

This is the right move because it diversifies along the type of trust, not just the count of participants. Adding more participants of the same type (back to the multi-participant VRF idea) brings the validator-network overhead with it. Adding more types of sources — some decentralised-federated, some centralised-but-credible, some operator, some public — buys resilience without that overhead.

So we have four channels: operator, public pool, drand, NIST. The combined seed will be honest as long as *at least one of these four* is honest in any given round. Three out of four can be adversarial; the protocol still produces an unpredictable, unbiassable seed.

4 Chapter 4: putting it together — the protocol step by step

Now let’s walk through what a single round actually does. Let R be the round number; the protocol runs on a fixed schedule, with each round consisting of a commit window, a reveal window, a beacon-pulling moment, and a final combine step.

Step 1: the commit phase

The round opens at a publicly-known moment t_0 . During the commit window (call it T_c seconds), each contributor picks a fresh random value, computes a commitment, and publishes it.

The operator commits. The operator picks a uniformly random 32-byte value v_{op} and a fresh 32-byte nonce n_{op} , computes

$$c_{\text{op}} = H(v_{\text{op}} \parallel n_{\text{op}})$$

and publishes $(R, c_{\text{op}}, \text{“operator”})$ to the protocol’s audit log.

The pool commits. Anyone in the world can participate. Each pool contributor i picks their own v_i and n_i , computes $c_i = H(v_i \parallel n_i)$, and publishes $(R, c_i, \text{“pool”, sender}_i)$ to the audit log. The pool’s submission endpoint stays open for the duration of the commit window.

At time $t_0 + T_c$, the commit window closes. No more commitments accepted.

Step 2: the reveal phase

The reveal window opens (for T_r seconds). Every party who committed in step 1 now publishes their actual value and nonce.

The operator reveals. Publishes $(R, v_{\text{op}}, n_{\text{op}})$. Anyone can verify:

$$H(v_{\text{op}} \parallel n_{\text{op}}) \stackrel{?}{=} c_{\text{op}}$$

If yes, the operator’s contribution is accepted into the round. If no (mismatch, or no reveal at all), the operator is flagged — and the protocol’s fallback rules kick in (more on that below).

Pool contributors reveal. Each contributor publishes (R, v_i, n_i) . Same check: $H(v_i \parallel n_i) \stackrel{?}{=} c_i$. Pool reveals that don’t match their commitments, or that arrive after $t_0 + T_c + T_r$, are excluded. The remaining valid pool reveals form a set $\{v_1, v_2, \dots, v_k\}$.

Step 3: pull beacon pulses (with a critical timing constraint)

After both windows have closed, the protocol fetches a fresh drand pulse and a fresh NIST pulse. The pulses must come from rounds (in drand’s case) or sequence numbers (in NIST’s case) that *postdate* the commit window’s close.

Why postdate? Because if a beacon pulse from before the commit phase were used, a committer would have seen it and could have chosen their value to bias the final seed. Postdating means no contributor knew the beacon value when they committed, so the beacon’s contribution is unbiassable from the contributors’ point of view.

Let $\text{pulse}_{\text{drand}}$ be the first drand pulse with round number $\geq r^*$, where r^* is the drand round corresponding to $t_0 + T_c$. Let $\text{pulse}_{\text{NIST}}$ be the first NIST pulse with timestamp $\geq t_0 + T_c$.

Both pulses come with cryptographic signatures (drand: threshold-BLS over its federation’s public key; NIST: standard signature over NIST’s public key). These signatures are verified against the beacons’ *publicly known* public infrastructure — not against anything the operator controls.

Step 4: aggregate the public pool

We need to combine the pool’s individual contributions into a single value before mixing it into the seed. The canonical rule is to **sort** the reveals and hash them:

$$\boxed{\text{pool_aggregate} = H(v_{(1)} \parallel v_{(2)} \parallel \dots \parallel v_{(k)})}$$

where $v_{(1)}, v_{(2)}, \dots, v_{(k)}$ are the valid reveals sorted by raw byte value (ascending, lexicographic, treating each as a fixed-length byte string).

Why sorted? Because hash functions are order-sensitive — $H(A \parallel B) \neq H(B \parallel A)$. If the protocol used arrival order, different verifiers (in different time zones, with different network latencies) might disagree about the order and compute different aggregates from the same set of contributions. Sorting by byte value is deterministic and order-independent: every honest party gets the same sorted sequence regardless of arrival order. It also closes a grinding attack where an aggregator could try many orderings looking for a final seed they prefer. Sorting removes the freedom to choose the ordering, so the attack vanishes.

Step 5: combine everything into the final seed

The four channels are all in. Now we hash them together under a fixed, public canonical rule. The exact recipe is:

$$\text{seed}_R = H\left(\text{DOMAIN_TAG} \parallel \text{uint64_be}(R) \parallel v_{\text{op}} \parallel \text{pool_aggregate} \parallel \text{pulse}_{\text{drand}} \parallel \text{pulse}_{\text{NIST}} \right)$$

The hash function is **SHAKE-256** with a 32-byte output. The pieces:

- **DOMAIN_TAG** is a fixed 16-byte string (e.g., "r`cs-vrf-seed-v1`", null-padded). Its job is **domain separation**: it ensures that this hash output can never be confused with hash outputs the protocol computes for other purposes (e.g., circuit generation, audit log linkage). Without a domain tag, an attacker could in principle craft inputs in another part of the protocol that collide with the seed computation.
- **uint64_be**(R) is the round number, encoded as 8 bytes in big-endian byte order. Including it guarantees that even if all other inputs were somehow identical across two rounds (astronomically unlikely but conceivable), the seeds would still differ. This defeats replay attacks.
- v_{op} is the 32-byte operator value (revealed in step 2).
- **pool_aggregate** is the 32-byte canonical hash of sorted pool reveals (from step 4).
- **pulse_{drand}** is the 32-byte drand pulse value (with its signature already verified in step 3).
- **pulse_{NIST}** is the 64-byte NIST pulse value.

The output is 32 bytes — a 256-bit seed, the same size and role as the EC-VRF seed it replaces in our protocol. The rest of the pipeline (Leg 3's circuit generation, the quantum sampling, verification, extraction, anchoring) does not need to change at all: the seed slots into Leg 3's deterministic SHAKE-256 expansion exactly as before.

Step 6: anchor in the audit log

Every piece of information from steps 1–5 is recorded in the protocol's tamper-evident audit log: every commitment, every reveal (matched against its commitment), the two beacon pulses with their signatures, the canonical combining rule, and the final seed. This is what makes the round verifiable after the fact.

5 Chapter 5: how verification works in the new design

In the old EC-VRF design, verifying a seed meant validating a single cryptographic proof: take (operator's public key, input, output, proof), run the VRF-verify algorithm, get a yes/no answer. One check, one cryptographic primitive, one trust anchor (the operator's public key).

In the new design, verification is fundamentally different. There is *no single proof to check*. Instead, the verifier **rebuilds the seed from scratch** using inputs that are all independently publicly available. Let me walk through it.

The verifier's algorithm

Given a claimed seed_R and the audit log for round *R*, the verifier performs the following checks. If every check passes, the seed is verified.

1. **For every commit-reveal pair (c_i, v_i, n_i) in the log:**

$$H(v_i \parallel n_i) \stackrel{?}{=} c_i.$$

This catches anyone who claimed to commit to one value and revealed another.

2. **For the drand pulse:**

- Fetch the pulse from drand independently (the verifier connects to drand directly, not via the operator).
- Verify the threshold-BLS signature against drand's published public key.
- Confirm the pulse's round number postdates the commit window close.

3. **For the NIST pulse:**

- Fetch from NIST independently.
- Verify NIST's signature against NIST's published public key.
- Confirm the pulse's timestamp postdates the commit window close.

4. **Recompute the pool aggregate:**

$$\text{pool_aggregate}_{\text{check}} = H(\text{sorted}(v_1, v_2, \dots, v_k)).$$

5. **Recompute the final seed:**

$$\text{seed}_{\text{check}} = H(\text{DOMAIN_TAG} \parallel \text{uint64_be}(R) \parallel v_{\text{op}} \parallel \text{pool_aggregate}_{\text{check}} \parallel \text{pulse}_{\text{drand}} \parallel \text{pulse}_{\text{NIST}}).$$

6. **Final equality check:**

$$\text{seed}_{\text{check}} \stackrel{?}{=} \text{seed}_R.$$

If all six steps pass, the round is verified. If any fails, the round is rejected.

Why this is a stronger form of verifiability than EC-VRF's

In EC-VRF, verifiability rests on *one* mathematical assumption (elliptic-curve discrete log) and *one* trust anchor (the operator's public key). If the math assumption breaks (e.g., a quantum attacker), or the operator's key is compromised, verification can pass when it shouldn't.

In the new design, verifiability rests on a *conjunction of independent assumptions*:

- **Hash function security.** The protocol depends on SHAKE-256 being collision-resistant and preimage-resistant. These properties are well-studied and quantum-resistant (Grover’s algorithm gives only a quadratic speedup, easily compensated by doubling output length).
- **drand’s threshold-BLS federation.** A majority of drand’s operators must be honest — which is itself a federated, independently-checkable assumption.
- **NIST’s signing integrity.** NIST must publish honest pulses (and not collude with us). This is checkable: NIST’s pulses are publicly verifiable by anyone.
- **The public audit log’s integrity.** Anyone watching the protocol can record the commitments at submission time and compare them to the audit log later.

The key shift: in EC-VRF, you trust the operator’s claim (validated by checking a proof they hand you). In commit-reveal + beacons, you don’t trust anyone’s claim — you re-fetch every input from its independent source and recompute the seed yourself. The operator never has to be believed; they just have to be checkable. And the checks are spread across four independent trust anchors instead of one.

6 Chapter 6: the role swap

Looking at the whole picture, the most important change between the old design and the new design isn’t really a change of cryptographic primitive (EC-VRF $\rightarrow$ hash functions). It’s a change of *workflow*. The redesign isn’t “use a different signature scheme”; it’s “use a different way of producing the result.”

The old workflow: request-then-fulfill

1. A user publicly requests randomness.
2. One operator sees the request, computes the result locally using their secret key.
3. There is a private window where the operator alone knows the result.
4. The operator publishes the result and proof.
5. Everyone sees the result for the first time.

The defining feature: **one party knows the result before everyone else**. That asymmetry is what creates the MEV surface.

The new workflow: schedule-driven commit-reveal

1. The protocol runs on a fixed schedule; rounds open and close on a timer, independent of any external request.
2. Multiple parties commit to values *before* any reveals — nobody knows anyone else’s value yet, including the operator.
3. Beacon pulses arrive on a fixed external schedule, postdating commitments, generated by parties outside the protocol.

4. Reveals happen openly in a fixed window; all reveals are visible to everyone simultaneously.
5. The final seed is computed by everyone, simultaneously, from all the public inputs.

The defining feature: **no party ever holds a computed-but-undelivered result**. The result materialises publicly the instant the last input is in. There is no private window, no privileged knowledge, nothing to front-run.

Why this kills MEV at the root

In the old design, MEV exists because of the time gap between “operator computes” and “operator publishes.” Closing that gap requires changing the workflow, not the cryptography. PQ-VRF can’t close the gap because it preserves the workflow. Commit-reveal + beacons closes the gap because it *eliminates the step where one party computes alone*. There is no alone-with-the-answer moment.

A useful way to feel the difference. In request-then-fulfill, the protocol’s output is something an operator delivers. In commit-reveal + beacons, the protocol’s output is something the world computes simultaneously from publicly-pinned inputs. Delivery is a privileged act; computation is not. By swapping “delivery” for “computation,” you remove the privilege.

7 Chapter 7: what’s left to pin down

A clean redesign deserves a clean list of remaining design choices. Most of these are not research questions; they are spec decisions to be made between people in a meeting room (which is exactly what the upcoming design meeting with Mehdi is for).

Open question 1: commit-reveal ordering

The relative timing of the commit window, the reveal window, and the beacon pulse drawing must be pinned down precisely. The constraint is: *the beacon pulses must postdate the commit window’s close*, otherwise a committer could see them and bias their value. The exact mechanism — block heights, signed wall-clock timestamps, drand round numbers anchored to a chain — is a spec decision.

Open question 2: the canonical combining rule

The strawman recipe in Chapter 4 gives the shape, but the exact byte-level details have to be frozen: which hash function (SHAKE-256 is the leading candidate), the domain tag string, fixed lengths for each field, byte order (big-endian is standard), whether to length-prefix variable-length fields. The goal: any two independent implementations produce byte-identical input to the hash, so every verifier gets the same answer.

Open question 3: fallback rules

What does the protocol do when something fails?

- Operator fails to reveal after committing $\Rightarrow$ likely abort the round entirely (the operator’s contribution is privileged enough that without it, the round is incomplete).

- Pool member fails to reveal $\Rightarrow$ exclude them from the aggregate, proceed.
- drand has an outage $\Rightarrow$ options: proceed with NIST + operator + pool only; wait; abort.
- NIST has a delay (it sometimes does) $\Rightarrow$ similar options.
- Public pool is empty in a given round $\Rightarrow$ proceed with operator + beacons; the security property degrades to “honest as long as operator OR drand OR NIST is honest,” still acceptable.

These rules need to be specified explicitly and tested, not handwaved.

Open question 4: public-pool aggregation details

If pool reveals are aggregated by sorted-hash (the choice in the strawman), the protocol must be defined down to byte ordering. If by Merkle root (an alternative that supports succinct membership proofs), the tree structure and hash conventions must be pinned. The choice between sorted-hash and Merkle is a tradeoff: sorted-hash is simpler, requires downloading all reveals to verify; Merkle is more complex, supports light verification at scale. At our current scale, sorted-hash is fine.

8 Chapter 8: where this leaves us

Stepping back: the seed-generation step started life in v0.2 as a real Ed25519 EC-VRF, which solved the obvious version of the seed problem with the textbook tool. On closer reading — and especially in conversation with Mehdi — we recognised that the textbook tool brought three problems with it, and that one alternative construction (hash-based commit-reveal augmented by external beacons) addresses all three at once.

The redesign:

- **Is quantum-resistant** — uses only hash functions and external beacons, no elliptic-curve primitives.
- **Eliminates the single-keyholder dilemma** — no party holds a secret key whose secrecy the seed depends on; no validator network needed.
- **Eliminates the MEV surface** — replaces request-then-fulfill with schedule-driven commit-reveal; no party ever holds a computed-but-undelivered result.
- **Strengthens verifiability** — replaces “check one proof against one public key” with “re-fetch and re-compute from four independent public sources.”
- **Reduces overhead** — no registration, no Sybil resistance, no incentive layer, no slashing infrastructure. Just commit, reveal, fetch, hash.

The new construction’s place in the larger protocol is exactly where the VRF used to be: it produces the 32-byte seed that drives Leg 3’s circuit generation. The rest of the pipeline doesn’t have to change. What changes is what happens *before* Leg 3 — a clean, schedule-driven, hash-based, beacon-augmented, MEV-immune, quantum-resistant ceremony for producing the spark that lights the engine.

One last picture, for the very end

Imagine the seed as a torch that gets lit at the beginning of each round. Under the old design, one runner produces the flame in private, then carries it to the start line. Whoever observes the runner during that private moment has an advantage. Under the new design, four people — the operator, the public crowd, a council of distant lighthouse-keepers (drand), and the national observatory (NIST) — each strike a spark independently, into a brazier whose contents only catch fire when the last spark arrives. By the time anyone can see what the flame looks like, every contribution is already in. There is no private moment. There is no privileged knowledge. The torch was always going to look exactly the way the public inputs determined — and everyone can see what those inputs were, and anyone can light their own torch by the same recipe and check that it matches.

That's the redesign.

Appendix: pseudocode

For completeness, here is the protocol expressed as pseudocode. This is the strawman; the design meeting with Mehdi will pin down the exact spec.

```
CONSTANTS:
    DOMAIN_TAG      = "rcs-vrf-seed-v1\0"
    HASH             = SHAKE256
    OUTPUT_LEN      = 32 bytes
    COMMIT_WINDOW   = T_c seconds
    REVEAL_WINDOW   = T_r seconds

PROCEDURE participate(round_id):
    # Each contributor (operator, each pool member) runs this independently.

    value = secure_random(32 bytes)
    nonce = secure_random(32 bytes)
    commit = HASH(value || nonce, OUTPUT_LEN)
    publish(round_id, commit, sender = me)

    wait_until(commit_window_closes(round_id))

    publish(round_id, value, nonce, sender = me)

PROCEDURE generate_seed(round_id):
    # Run by anyone applying the canonical rule.

    # 1. Read all commitments and reveals from the audit log.
    commitments = audit_log.commitments(round_id)
    reveals     = []
    for (sender, commit) in commitments:
        (value, nonce) = audit_log.reveal(round_id, sender)
        if HASH(value || nonce, OUTPUT_LEN) != commit:
            apply_fallback(sender)
            continue
        reveals.append((sender, value))
```

```

# 2. Separate the operator's value from the pool reveals.
operator_value = reveals.where(sender == OPERATOR_PUBKEY).value
pool_values    = sorted([v for (s, v) in reveals if s != OPERATOR_PUBKEY])
pool_aggregate = HASH(concat(pool_values), OUTPUT_LEN)

# 3. Fetch beacon pulses that postdate the commit phase.
commit_close = commit_window_close_time(round_id)
drand_pulse  = drand.fetch_after(commit_close)
nist_pulse   = nist.fetch_after(commit_close)

assert drand.signature_valid(drand_pulse)
assert nist.signature_valid(nist_pulse)

# 4. Combine under the canonical rule.
seed = HASH(
    DOMAIN_TAG           ||
    uint64_be(round_id)   ||
    operator_value       || # 32 bytes
    pool_aggregate       || # 32 bytes
    drand_pulse.randomness || # 32 bytes
    nist_pulse.output_value, # 64 bytes
    OUTPUT_LEN
)

# 5. Anchor everything in the audit log.
audit_log.record(round_id, {
    commitments, reveals,
    drand_pulse, nist_pulse,
    seed
})

return seed

```

```

PROCEDURE verify_seed(round_id, claimed_seed):
    # Anyone, any time after the round, can run this.

    log = audit_log.fetch(round_id)

    # Re-check every commit-reveal pair.
    for (sender, commit) in log.commitments:
        (value, nonce) = log.reveal(sender)
        assert HASH(value || nonce, OUTPUT_LEN) == commit

    # Re-aggregate the pool.
    pool_aggregate = HASH(
        concat(sorted([v for (s, v) in log.reveals if s != OPERATOR_PUBKEY])),
        OUTPUT_LEN
    )

    # Re-verify beacon pulses against their own public infrastructure.
    assert drand.signature_valid(log.drand_pulse)

```

```

assert drand.round_postdates(log.drand_pulse, log.commit_window_close_time)
assert nist.signature_valid(log.nist_pulse)
assert nist.timestamp_postdates(log.nist_pulse, log.commit_window_close_time)

# Recompute the seed under the canonical rule.
operator_value = log.reveals.where(sender == OPERATOR_PUBKEY).value
recomputed = HASH(
    DOMAIN_TAG                ||
    uint64_be(round_id)         ||
    operator_value              ||
    pool_aggregate              ||
    log.drand_pulse.randomness  ||
    log.nist_pulse.output_value,
    OUTPUT_LEN
)

return recomputed == claimed_seed

```